

HARDWARE DESIGN DOCUMENT

電子工作実装設計書

fighter-game × Raspberry Pi 4
GPIOボタン・NeoPixel LED・加速度センサー連携

対象機材	Raspberry Pi 4 (Raspberry Pi OS 64bit)
対象ソフト	fighter-game (素のHTML + CSS + JavaScript)
予算目安	2,500 ~ 4,000円
版	第1版
作成日	2026年9月12日

1. 概要

スマブラ風対戦ゲーム「fighter-game」に、GPIOボタン・NeoPixel LEDテープ・MPU-6050加速度センサーを接続する。画面の中だけで完結していた対戦に、手で押すボタンの感触と、当たった瞬間に光るLEDを足して、リアルな体験を追加することが目的である。

ゲーム本体（ブラウザで動くJavaScript）は書きかえを最小限にとどめ、電子部品の制御はすべてPython側の別プロセスに分離する。両者はWebSocketでつなぐ。この構成にすることで、電子部品をつながない状態でも、ゲームはこれまでどおりキーボードだけで遊べる。

重要な前提：ゲーム本体には外部ライブラリを一切追加しない。WebSocketクライアントはブラウザ標準のWebSocketを使う。Python側のライブラリ（RPi.GPIO / rpi-ws281x / smbus2 / websockets）は、ゲームとは別プロセスのため、この制約の対象外とする。

2. 必要部品

部品	数量	用途	参考価格
タクトスイッチ（6mm）	10個	1P / 2P 操作ボタン	100 ～ 300円
ブレッドボード（400穴）	2枚	配線	300 ～ 500円 / 枚
ジャンパーワイヤー（オス-メス）	40本	GPIO接続	300 ～ 500円
WS2812B NeoPixel LEDテープ 1m（60LED/m）	1本	ヒットエフェクト	800 ～ 1,500円
MPU-6050 GY-521 モジュール	2個	加速度センサーコントローラー	200 ～ 400円 / 個
抵抗 470Ω	1個	NeoPixel データライン保護	数円
コンデンサ 1000μF	1個	NeoPixel 電源ノイズ除去	数十円
合計目安			2,500 ～ 4,000円

3. システムアーキテクチャ

```
ブラウザ (Chromium)
| WebSocket (ws://localhost:8765)
▼
hardware_server.py (Python)
├─ GPIO入力 ── タクトスイッチ (1P/2P)
│               → xdotool でキーイベント注入
├─ GPIO出力 ── NeoPixel LEDテープ
│               → ゲームイベントで発光
└─ I2C ──── MPU-6050 × 2
              → 傾き / 振動 → 移動 / 攻撃
```

ゲーム側からPython側へは、hit（攻撃が当たった）・ko（撃墜した）・damage（ダメージ変化）の3種類のイベントをJSONで送る。Python側からゲーム側へは、入力をxdotool経由のキーイベントとして届けるため、WebSocketでの送り返しは行わない。

4. GPIO ピン割り当て

4.1 1P ボタン（左手側）

ボタン	GPIO	物理ピン
左移動	2	3
右移動	3	5
ジャンプ	4	7
しゃがみ	17	11
弱攻撃	27	13
横強攻撃	22	15

4.2 2P ボタン（右手側）

ボタン	GPIO	物理ピン
左移動	10	19
右移動	9	21
ジャンプ	11	23
しゃがみ	5	29
弱攻撃	6	31
横強攻撃	13	33

4.3 NeoPixel

信号	GPIO	物理ピン
DATA	18	12
VCC	5V	2
GND	GND	6

4.4 MPU-6050 (I2C)

信号	GPIO	物理ピン / 備考
SDA	2 (I2C)	物理ピン 3
SCL	3 (I2C)	物理ピン 5
1P AD0	LOW (GND)	→ アドレス 0x68
2P AD0	HIGH (3.3V)	→ アドレス 0x69

ピン競合の注意：1Pボタンの「左移動」(GPIO 2)「右移動」(GPIO 3) は、MPU-6050 の SDA / SCL と同じ物理ピン (3番・5番) を使う。ボタンと加速度センサーを同時に使うことはできない。同時に使いたい場合は、1Pの左右移動を空いているGPIO (例：23番・24番) へ移すこと。

5. 配線概略図

ラズパイ 3.3V	MPU1 VCC, MPU2 VCC
ラズパイ GND	MPU1 GND, MPU2 GND, NeoPixel GND, ボタン共通GND
ラズパイ GPIO18 -[470Ω]	NeoPixel DATA
ラズパイ 5V -[1000μF]	NeoPixel VCC
ラズパイ SDA(GPIO2)	MPU1 SDA, MPU2 SDA
ラズパイ SCL(GPIO3)	MPU1 SCL, MPU2 SCL
MPU1 AD0	GND (アドレス 0x68)
MPU2 AD0	3.3V (アドレス 0x69)
各ボタン	GPIOピン ~ GND (内部プルアップ使用)

コンデンサ1000μFは NeoPixel の VCC と GND のあいだに入れる。LEDが一気に光るときの電流の波をならし、ラズパイ本体が電圧不足で落ちるのを防ぐためである。抵抗470Ωは、ラズパイのデータ線とLEDテープのあいだに直列に入れて、信号の反射と過電流からGPIOを守る。

6. 実装フェーズ

PHASE 1 WebSocket基盤

- ゲームのJavaScriptにWebSocketクライアントを追加する (ws://localhost:8765)
- Pythonサーバー (websockets) は、ゲーム配信用のHTTPサーバーとは**別プロセス**で起動する
- ゲームイベント (hit / ko / damage) をJSONで送信する
- 接続できなかった場合もゲームは通常どおり動くこと (接続失敗を無視する)

PHASE 2 GPIO ボタン

- RPi.GPIO でボタン入力を読み取る (内部プルアップ、押すとLOW)
- xdotool でブラウザにキーイベントを注入する
- hardware_server.py に組み込む
- チャタリング対策として、同じボタンの再検出は20ms以上あけること

PHASE 3 NeoPixel LED

- rpi-ws281x ライブラリで制御する
- hit → 1秒間点滅 (被ダメージ側の色。1P=青 / 2P=赤)
- ko → 虹色フラッシュ3回
- 試合進行中は、テープの両端が残りストック数に応じて点灯する

PHASE 4 MPU-6050 加速度センサー

- smbus2 で I2C 読み取り (16ms周期 = 約60回/秒。ゲームの1フレームに対応)
- X軸の傾き → 移動 (±15度をこえたら入力とみなす)
- 振り (加速度スパイク検出) → 攻撃
- ボタンとの併用を可能にする (センサーを持つコントローラーとして使う想定)

7. ゲーム側パラメータとの対応

LEDの光りかたや判定のしきい値など、数値はすべて `src/config.js` に定数として集約する方針に合わせる。
Python側で持つ数値は、次の表のとおりゲーム側の仕様と対応させること。

項目	値	根拠 / 対応するゲーム仕様
センサー読み取り周期	16 ms	ゲームが固定タイムステップ60fpsのため
傾き入力のしきい値	±15 度	地上移動速度 5 に対応させる
ヒット時LED点滅	1 秒	ヒットストップ（弱5F / 横強8F）より長く見せる
ボタン再検出の間隔	20 ms	1フレーム（約16.7ms）より長くする
1P / 2P の色	青 / 赤	ゲーム画面の 1P=青、2P=赤 と合わせる

8. 動作確認の手順

- **Phase 1** : ブラウザのコンソールに `WebSocket connected` が出る。攻撃を当てるとPython側のログに `hit` が流れる
- **Phase 2** : `gpio readall` で各ピンの状態を確認。ボタンを押すと該当ピンが `1 → 0` に変わる
- **Phase 3** : LEDテープ単体テストを先に通す。全消灯 → 全点灯 → 虹色の順で確認する
- **Phase 4** : `i2cdetect -y 1` で 68 と 69 の両方が見えること

9. 参考リンク

- BTF-LIGHTING WS2812B 5m 60LED/m
<https://www.amazon.co.jp/dp/B088K61XYW>
- MPU-6050 GY-521 モジュール
<https://www.amazon.co.jp/dp/B081RHK82T>